Mini-project 2 in Advanced Machine Learning (02460)

by Lukas W. Falk Nyholm (s214426) on 3 April 2025

Part A: pullback geodesics

A plot of the latent variables with the geodesics of 25 random latent variable pairs, can be seen in Figure 1.

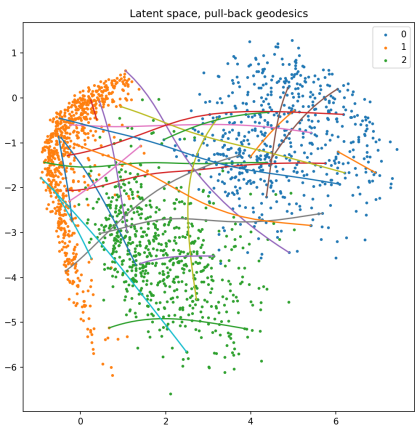

Figure 1: Geodesics between 25 randomly sampled latent variable pairs.

The code of the algorithms used for computing the energy of a curve and computing geodesics, was from previous exercise solutions. `curve_energy()` can be seen in Listing 1 and `connecting_geodesic()` can be seen in Listing 2.

The computed geodesics are similar across runs. The geodesics reliably connects the different classes, although they change every run, but that's to be expected due to VAEs being trained with random initializations. Inspecting the plot, geodesics tend to stay within class clusters.

Part B: Ensemble VAE geometry

The plot of the latent variables with geodesics between the same random latent variable pairs, as in Part A, can be seen in Figure 2.

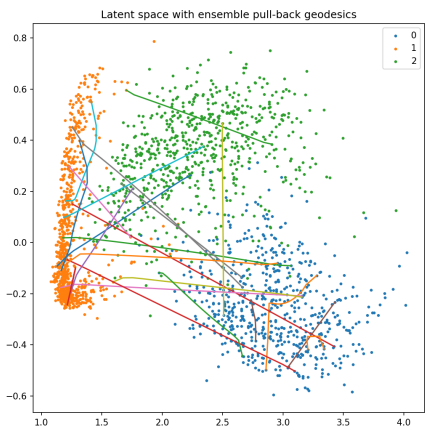

Figure 2: Latent variables with geodesics, using the same random latent variable pairs as in Part A, but using an ensemble VAE.

The geodesics are straighter in their curvature, showing that the geodesics lies on the paths of what the ensemble VAEs agrees on. The geodesics tend to stay more within the more dense regions of the latent variables, instead of going through low-density space, which would give nonsensical results.
The latent space itself looks more separated between the three classes as well.

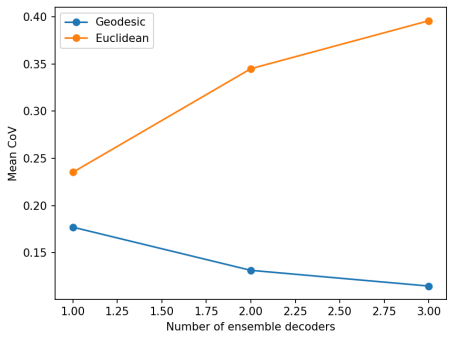

Figure 3: The coefficient of variation across multiple training runs for different numbers of decoders.

From Figure 3, it can be seen that the Mean CoV increases with the number of ensemble decoders when doing Euclidean distancing.
On the other hand, using the geodesics, the CoV gets better with more ensemble decoders. This makes sense, as the geodesics find the best curve in the latent space, as opposed to Euclidean, finding curves through the low-density regions of the latent space.
The functions `curve_energy()` and `connecting_geodesic()` had to be adapted to the ensemble VAEs, and can be found in Listing 3 and 4. The `CoV()` function is in Listing 5.
To overcome the hurdle of tuning the optimizer for ensemble VAEs, I clipped the gradients in `connecting_ensemble_geodesic()` and set the learning rate to 0.001.

Ultimately, ensemble VAEs do a better job, getting more consistent results that reduce model uncertainty, resulting in a lower mean CoV, than single VAEs.

Code snippets

Part A

```python
def curve_energy(metric, curve):
    G = metric(curve[:-1])  # (N-1)xDxD
    delta = curve[1:] - curve[:-1]  # (N-1)xD
    tmp = torch.bmm(G, delta.unsqueeze(-1)).squeeze(-1)
    energy = torch.sum(delta * tmp)
    return energy
```

Listing 1: Energy of a curve

```python
def connecting_geodesic(metric, curve):
    opt = optim.LBFGS([curve.params], lr=0.5)
    def closure():
        opt.zero_grad()
        energy = curve_energy(metric, curve.points())
        energy.backward()
        return energy
    max_iter = 100
    for i in range(max_iter):
        opt.step(closure)
```

Listing 2: Computing geodesics

Part B

```python
def ensemble_curve_energy(decoders, curve, n_samples=10):
    energy = 0
    M = len(decoders)
    for _ in range(n_samples):
        l = torch.randint(M, (1,)).item()
        k = torch.randint(M, (1,)).item()
        f_l = decoders[l].decoder_net(curve[:-1])  # (N-1, 1, 28, 28)
        f_k = decoders[k].decoder_net(curve[1:])   # (N-1, 1, 28, 28)
        diff = (f_l - f_k).flatten(1)              # (N-1, 784)
        energy = energy + torch.sum(diff**2)
    return energy / n_samples
```

Listing 3: Energy of a curve using ensemble VAE

```python
def connecting_ensemble_geodesic(decoders, curve, n_samples=5):
    opt = optim.LBFGS([curve.params], lr=0.001, max_iter=20,
                      tolerance_grad=1e-3, tolerance_change=1e-4,
                      line_search_fn='strong_wolfe')
    def closure():
        opt.zero_grad()
        energy = ensemble_curve_energy(decoders, curve.points(), n_samples)
        energy.backward()
        torch.nn.utils.clip_grad_norm_([curve.params], max_norm=1.0)
        return energy

    for _ in range(20):
        opt.step(closure)
```

Listing 4: Computing geodesics

```python
def CoV(dist_key, all_distances):
    covs = []
    for decoder_number in range(1, 4): # [1, 2, 3]
        d = all_distances[decoder_number][dist_key]  # (reruns, curves)
        # CoV per pair
        cov_per_pair = d.std(dim=0) / d.mean(dim=0)  # (curves)
        # Average CoV across pairs
        covs.append(cov_per_pair.mean().item())
    return covs
```

Listing 5: Coefficient of Variation

References